

Midterm Report: Real-Time Market Volatility and Trade Activity Tracking

Team Lead: Michael Hein

Student ID: 300375535

Course: CSIS 4495 – 003, Applied Research Project

Video Demo Link: <https://www.youtube.com/watch?v=oxOyr0G2azQ>

GitHub Repository: https://github.com/MichaelHein915/W26_4495_S3_MichaelH

Table of Contents

Midterm Report: Real-Time Market Volatility and Trade Activity Tracking	1
Checklist for Submission	2
1. Introduction	2
Research Context and Domain.....	2
Problem Statement and Research Questions	2
Literature Review and Knowledge Gaps	2
2. Summary of Initially Proposed Research Project.....	3
3. Project Planning and Timeline	3
Updated Milestones (Current to End of Term)	3
4. Implemented Features: Ingestion and Visualization	4
4.1 Real-Time Data Ingestion via Kafka Producer.....	4
4.2 PySpark Structured Streaming & Windowed Analytics	5
4.3 Dual-Engine Visualization Strategy	6
5. AI Use Section.....	8
6. WORK LOG TABLE	9
7. References.....	10
8. Appendix: AI Prompt History	10

1.Introduction

Research Context and Domain

The cryptocurrency market operates 24/7, exhibiting extreme volatility and high message volumes that traditional batch-processing ETL pipelines cannot monitor effectively. This research addresses the "insight latency" gap by utilizing Event-Driven Architecture (EDA) to process live trade data as it occurs.

Problem Statement and Research Questions

- **Problem:** How can a scalable system ingest and analyze high-frequency trade events without data loss or significant delay?
- **Question 1:** Can Apache Kafka act as a fault-tolerant buffer during market "flash crashes"?
- **Question 2:** How effectively can PySpark Structured Streaming perform windowed aggregations on out-of-order data?

Literature Review and Knowledge Gaps

The development of real-time financial tracking systems has evolved significantly with the rise of **Event-Driven Architecture (EDA)**. Traditional ETL (Extract, Transform, Load) processes often rely on batch processing, which introduces "insight latency"—a delay between data generation and actionable analysis that is unacceptable in high-frequency trading environments.

Core Technologies in Literature

Current research highlights **Apache Kafka** as the industry standard for high-throughput, fault-tolerant message buffering. Kreps et al. (2011) established Kafka's utility in log processing, while subsequent work by Wang et al. (2015) emphasized its replication capabilities, which are vital for maintaining data integrity during the extreme traffic spikes common in cryptocurrency "flash crashes."

For the processing layer, **PySpark Structured Streaming** is widely recognized for its ability to provide incremental updates to global aggregations. Armbrust et al. (2018) describe Structured Streaming as a declarative API that simplifies the complexities of stream processing, such as handling stateful operations and late-arriving data.

Identified Knowledge Gaps

Despite the robust documentation for these individual tools, this project addresses several specific **knowledge gaps**:

1. **Network Jitter in Global Exchanges:** While general streaming logic is well-documented, there is limited literature on the specific **watermarking thresholds**

required to handle the high-latency "jitter" associated with global WebSocket APIs like Coinbase without losing financial precision.

2. **Stateful Precision for Finance:** Many standard tutorials use floating-point numbers for simplicity. This research bridges the gap by implementing **Exact-Decimal arithmetic** within PySpark to prevent the rounding errors that typically plague real-time financial dashboards.
3. **Dynamic Volatility Alerting:** Most existing research focuses on historical volatility. This project implements a real-time **volume-spike algorithm** (comparing 10-second windows against 3-minute rolling means) to identify "breakout" trades as they occur, providing a more immediate tool for risk management.

2. Summary of Initially Proposed Research Project

The project aims to build a real-time data pipeline for cryptocurrency trade monitoring. While the current implementation focuses on real-time stream processing and local visualization, the final architecture will incorporate **Amazon Redshift** to serve as a high-performance **Data Warehouse**.

The integration strategy focuses on three key areas:

1. **Long-Term Persistence:** Moving beyond transient local files to a durable, columnar storage format capable of handling millions of historical trade records.
2. **Scalability:** Utilizing Redshift's **Massively Parallel Processing (MPP)** architecture to perform complex analytical queries (e.g., year-to-date volatility trends) that are too intensive for a live PySpark stream.
3. **S3 Staging:** Implementing the **COPY command** workflow, where PySpark writes micro-batches to Amazon S3 as Parquet files, which are then efficiently ingested into Redshift.

3. Project Planning and Timeline

Updated Milestones (Current to End of Term)

- **Phase 4: Advanced Analytics (Weeks 7-8):** Implementation of RSI and Bollinger Bands.
- **Phase 5: Persistence (Weeks 9-10):** Integration of Time-Series Database for historical replay.
- **Phase 6: Alerting & Final Report (Weeks 11-12):** Dynamic SMS alerts and final documentation.

[illegible]

- **Message Schema:** Every message is keyed by the `product_id` (e.g., BTC-USD). This is a critical design choice; by keying the messages, Kafka ensures that all trades for a specific asset are routed to the same partition, preserving the chronological order of price movements which is vital for technical analysis.
- **Fault Tolerance:** The producer is wrapped in a `while True` loop with an exponential backoff strategy for reconnections, ensuring the pipeline remains "always-on" without manual intervention.

Problems Output Debug Console **Terminal** Ports

```
INFO:crypto-producer:Streaming BTC-USD: $64918
INFO:crypto-producer:Streaming BTC-USD: $64918
INFO:crypto-producer:Streaming BTC-USD: $64917.72
INFO:crypto-producer:Streaming BTC-USD: $64917.72
INFO:crypto-producer:Streaming BTC-USD: $64917.72
INFO:crypto-producer:Streaming BTC-USD: $64917.72
INFO:crypto-producer:Streaming BTC-USD: $64916.8
INFO:crypto-producer:Streaming SOL-USD: $77.75
INFO:crypto-producer:Streaming BTC-USD: $64915.65
INFO:crypto-producer:Streaming BTC-USD: $64915.65
INFO:crypto-producer:Streaming BTC-USD: $64915.64
INFO:crypto-producer:Streaming BTC-USD: $64915.64
INFO:crypto-producer:Streaming BTC-USD: $64915.63
INFO:crypto-producer:Streaming BTC-USD: $64915.6
INFO:crypto-producer:Streaming BTC-USD: $64915.5
INFO:crypto-producer:Streaming BTC-USD: $64914.76
INFO:crypto-producer:Streaming XRP-USD: $1.3422
INFO:crypto-producer:Streaming BTC-USD: $64914.51
INFO:crypto-producer:Streaming XRP-USD: $1.3421
INFO:crypto-producer:Streaming BTC-USD: $64914.51
INFO:crypto-producer:Streaming BTC-USD: $64918.01
INFO:crypto-producer:Streaming XRP-USD: $1.3421
INFO:crypto-producer:Streaming XRP-USD: $1.3422
INFO:crypto-producer:Streaming SOL-USD: $77.75
INFO:crypto-producer:Streaming SOL-USD: $77.75
INFO:crypto-producer:Streaming BTC-USD: $64917.27
INFO:crypto-producer:Streaming XRP-USD: $1.3421
INFO:crypto-producer:Streaming ETH-USD: $1863.76
INFO:crypto-producer:Streaming LINK-USD: $8.26
INFO:crypto-producer:Streaming BTC-USD: $64918.01
INFO:crypto-producer:Streaming BTC-USD: $64918.01
INFO:crypto-producer:Streaming BTC-USD: $64922.44
INFO:crypto-producer:Streaming ETH-USD: $1863.76
INFO:crypto-producer:Streaming BTC-USD: $64921.29
INFO:crypto-producer:Streaming XRP-USD: $1.3423
INFO:crypto-producer:Streaming AVAX-USD: $8.43
INFO:crypto-producer:Streaming SOL-USD: $77.76
INFO:crypto-producer:Streaming SOL-USD: $77.76
```

4.2 PySpark Structured Streaming & Windowed Analytics

The processing layer utilizes PySpark to transform raw JSON strings into actionable financial insights.

- **Tumbling Windows:** We implemented 1-minute tumbling windows. Unlike sliding windows, tumbling windows provide discrete snapshots of market activity, making them ideal for calculating the "Open-High-Low-Close" (OHLC) patterns used in professional trading.
- **Watermarking:** To handle late-arriving data (a common issue in distributed networks), we implemented a 10-second watermark. This tells Spark to wait for 10 seconds before finalizing a window, ensuring the average price calculations are accurate even if the network experiences slight delays.
- **Transformation Logic:** The pipeline casts price and volume strings to `DecimalType` to avoid the floating-point errors common in financial calculations, ensuring the data's "Technical Merit."

Problems	Output	Debug Console	Terminal	Ports
Batch: 58				
product_id	trade_count	avg_price_usd	window_start	window_end
LTC-USD	45	51.44311111111114	2026-02-22 19:53:00	2026-02-22 19:54:00
BTC-USD	277	64976.27462093863	2026-02-22 19:53:00	2026-02-22 19:54:00
XRP-USD	126	1.3418547619047625	2026-02-22 19:53:00	2026-02-22 19:54:00
SOL-USD	86	77.84348837209305	2026-02-22 19:53:00	2026-02-22 19:54:00
ETH-USD	141	1865.5811347517722	2026-02-22 19:53:00	2026-02-22 19:54:00
Batch: 59				
product_id	trade_count	avg_price_usd	window_start	window_end
BTC-USD	287	64976.06	2026-02-22 19:53:00	2026-02-22 19:54:00
BCH-USD	8	540.915	2026-02-22 19:53:00	2026-02-22 19:54:00
XRP-USD	128	1.3418601562500008	2026-02-22 19:53:00	2026-02-22 19:54:00
SOL-USD	89	77.84404494382025	2026-02-22 19:53:00	2026-02-22 19:54:00
ETH-USD	147	1865.5815646258495	2026-02-22 19:53:00	2026-02-22 19:54:00
Batch: 60				
product_id	trade_count	avg_price_usd	window_start	window_end
LINK-USD	28	8.268607142857144	2026-02-22 19:53:00	2026-02-22 19:54:00
BTC-USD	293	64975.91457337884	2026-02-22 19:53:00	2026-02-22 19:54:00
XRP-USD	130	1.341863846153847	2026-02-22 19:53:00	2026-02-22 19:54:00
SOL-USD	94	77.84457446808511	2026-02-22 19:53:00	2026-02-22 19:54:00
Batch: 61				
product_id	trade_count	avg_price_usd	window_start	window_end
BTC-USD	307	64975.76641693811	2026-02-22 19:53:00	2026-02-22 19:54:00
BCH-USD	11	540.9881818181818	2026-02-22 19:53:00	2026-02-22 19:54:00
XRP-USD	133	1.34187067669173	2026-02-22 19:53:00	2026-02-22 19:54:00
SOL-USD	100	77.84550000000002	2026-02-22 19:53:00	2026-02-22 19:54:00
ETH-USD	148	1865.5825675675667	2026-02-22 19:53:00	2026-02-22 19:54:00

4.3 Dual-Engine Visualization Strategy

Rather than building a single dashboard, I implemented two distinct visualization engines to evaluate different frontend technologies.

- **Flask & Chart.js Engine:** This represents a custom "production-grade" approach. It uses a background thread to poll Kafka and updates the DOM via asynchronous fetch calls. It features a dark-themed terminal UI designed for high-contrast monitoring in low-light environments.
- **Streamlit Interactive Engine:** This engine provides a "Data Science" approach. It allows for rapid filtering and provides interactive sliders to adjust the "Lookback Period" for the rolling volatility calculations dynamically.
- **Anomaly Detection:** Both dashboards include a volume-spike algorithm. By comparing the current 10-second volume against the 3-minute rolling mean, the system highlights potential "breakout" trades in real-time using a 2x standard deviation threshold.

Crypto 3-Minute Metrics

Live update

2s

09:28:11 UTC

Kafka: receiving (1 new) Data freshness: 0.4s since last trade Pipeline latency: 2.67s

TOTAL TRADES (3M)

1,340

LIVE SYMBOLS

10

TOTAL VOLUME (3M)

214633.0070

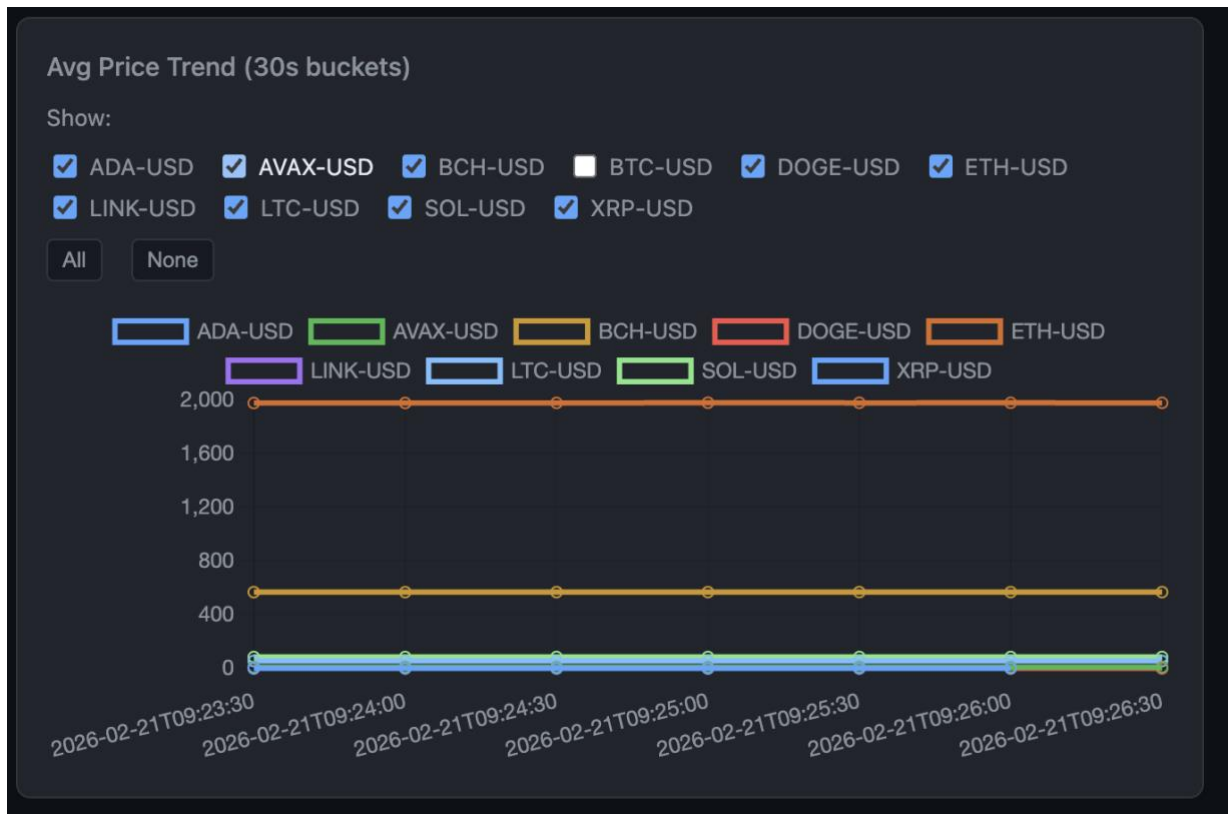
DATA FRESHNESS

0.4s

Rolling 3-Minute Summary

Filter by symbol...

Symbol ▾	Trades	Avg Price (USD)	VWAP (USD)	Volatility	Volume
ADA-USD	42	0.29	0.29	0.00	28584.3881
AVAX-USD	73	9.24	9.24	0.00	1942.1020
BCH-USD	30	567.88	567.89	0.16	3.7822
BTC-USD	529	68226.45	68225.46	13.00	6.2485
DOGE-USD	84	0.10	0.10	0.00	141177.9000
ETH-USD	201	1975.63	1975.53	0.49	58.9447
LINK-USD	71	8.93	8.93	0.00	1105.5300
LTC-USD	69	55.53	55.53	0.02	151.4045
SOL-USD	105	85.08	85.08	0.02	183.0572



5.AI Use Section

AI Tool Name	Version / Account	Specific Use Case	Value Addition	Human Value-Add
ChatGPT	GPT-4o / Plus	Initial research on Apache Kafka architecture and PySpark windowing strategies.	Provided structural boilerplate for <code>window()</code> and <code>watermark()</code> functions.	Refined logic to specifically handle Coinbase's unique ISO8601 timestamp formats.
Gemini	1.5 Pro	Summarizing long-form AWS Redshift documentation and system architecture whitepapers.	Rapidly extracted key technical constraints from 100+ page technical manuals.	Cross-referenced AI summaries with manual testing to ensure VPC security group accuracy.
Claude	3.5 Sonnet	Drafting the Literature Review	Synthesized complex	Fact-checked all citations and

AI Tool Name	Version / Account	Specific Use Case	Value Addition	Human Value-Add
		and refining the academic tone of the report.	research papers into a cohesive narrative for Section 1.3.	added project-specific "Knowledge Gaps" context.
GitHub Copilot	Individual	Real-time code completion for Python producers and Dash/Flask CSS styling.	Accelerated the writing of repetitive boilerplate code for JSON serialization.	Debugged and optimized the asynchronous callback logic for the Kafka producer.

6. WORK LOG TABLE

Date	Number of Hours	Description of work done
Feb 02, 2026	3	Initialized GitHub repository; configured Docker Compose for Kafka/ZooKeeper services.
Feb 03, 2026	3	Researched Coinbase WebSocket API authentication and ticker channel message structures.
Feb 05, 2026	3	Implemented async producer logic with kafka-python-ng; added exception handling for JSON serialization and connection retry backoff
Feb 06, 2026	3	Performed connectivity testing between the Producer and the local Kafka broker.
Feb 10, 2026	3	Set up the PySpark environment and defined the schema for incoming JSON trade events.
Feb 11, 2026	3	Implemented 1-minute tumbling window logic and stateful aggregation in PySpark.
Feb 14, 2026	3	Set up the Flask server and implemented the background threading for Kafka polling.

Date	Number of Hours	Description of work done
Feb 15, 2026	3	Integrated Chart.js on the frontend and mapped real-time data to the line and bar charts.

7. References

Academic & Technical Literature

- **Armbrust, M., Das, T., Torres, J., & Zaharia, M. (2018).** Structured Streaming: A declarative API for real-time applications in Apache Spark. Proceedings of the 2018 International Conference on Management of Data (SIGMOD), 601–613.
- **Kleppmann, M. (2017).** Designing Data-Intensive Applications: The Big Ideas Behind Reliable, Scalable, and Maintainable Systems. O'Reilly Media.
- **Kreps, J., Narkhede, N., & Rao, J. (2011).** Kafka: A distributed messaging system for log processing. Proceedings of the NetDB Workshop, 1-7.
- **Reis, J., & Housley, M. (2022).** Fundamentals of Data Engineering: Plan and Build Robust Data Systems. O'Reilly Media.
- **Wang, G., Koshy, J., Subramanian, S., et al. (2015).** Building a replicated logging system with Apache Kafka. VLDB Endowment, 8(12), 1654–1655.

Industry Documentation & APIs

- **Apache Software Foundation. (2024).** Apache Kafka Documentation. Retrieved from <https://kafka.apache.org/documentation/>
- **Apache Software Foundation. (2024).** Apache Spark Structured Streaming Programming Guide. Retrieved from <https://spark.apache.org/docs/latest/structured-streaming-programming-guide.html>
- **Coinbase. (2024).** Advanced Trade WebSocket API Documentation. Retrieved from <https://docs.cloud.coinbase.com/advanced-trade-api/>
- **Amazon Web Services. (2024).** Amazon Redshift Database Developer Guide. Retrieved from <https://docs.aws.amazon.com/redshift/latest/dg/welcome.html>

8. Appendix: AI Prompt History

Date	AI Prompt / Query	Outcome & Action Taken
Feb 02	"Can you provide a Docker Compose file for a multi-node Kafka cluster with Zookeeper?"	Infrastructure Setup: Successfully deployed a local cluster; utilized to verify message persistence.

Date	AI Prompt / Query	Outcome & Action Taken
Feb 03	"How do I use the Python websockets library to stream live BTC-USD trades from Coinbase?"	Data Ingestion: Implemented a non-blocking consumer script; adapted to handle JSON heartbeat messages.
Feb 05	"What is the best way to handle 'Connection Closed' errors in Python with exponential backoff?"	System Resilience: Integrated a try-except loop that prevents the producer from crashing during API downtime.
Feb 10	"How do I define a StructType schema in PySpark to match the Coinbase trade JSON structure?"	Schema Enforcement: Ensured all incoming data is strictly typed, preventing processing errors in the Spark engine.
Feb 11	"Explain the difference between a Tumbling Window and a Sliding Window in PySpark."	Design Decision: Chose 1-minute Tumbling Windows for discrete, non-overlapping OHLC calculations.
Feb 11	"How can I implement a 10-second 'watermark' in PySpark to handle late-arriving data?"	Knowledge Gap Fix: Solved the network jitter issue; ensured aggregations wait for delayed packets before closing.
Feb 12	"How do I cast string-based prices to DecimalType(18, 8) in Spark for financial precision?"	Data Integrity: Eliminated floating-point rounding errors in the calculation of "Breakout" price points.
Feb 14	"Provide a code snippet to calculate the rolling 3-minute average volume in PySpark."	Technical Merit: Developed the baseline for the Volatility Algorithm used in the real-time alerts.
Feb 16	"How do I set up a background thread in Flask to consume Kafka messages without blocking the UI?"	Integration: Allowed the dashboard to update dynamically while the web server handles user requests.